

Sure! Here are your **N-Queens Notes** in a clean, exam-friendly format that you can revise quickly.



N-Queens Problem Notes

1. Problem Statement

Given an $N \times N$ chessboard, place **N Queens** so that:

- ✓ No two queens are in the same row.
- ✓ No two queens are in the same column.
- ✓ No two queens are on the same diagonal.

2. Example (N = 4)

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```



Board Representation:

```
0 1 0 0  
0 0 0 1  
1 0 0 0  
0 0 1 0
```



- 1 → Queen
- 0 → Empty Cell

3. Concept Used

Backtracking

Definition

Backtracking is an algorithmic technique where we:

1. Try a choice.
2. Check if it is valid.
3. Move to the next step.
4. If stuck, undo the choice and try another option.

Memory Trick

```
Place Queen
  ↓
Check Safe?
  ↓
Yes
  ↓
Go to Next Row
  ↓
No Solution?
  ↓
Backtrack (Remove Queen)
  ↓
Try Next Column
```



4. Global Variables

</> Java



```
static int n;
static int[][] board;
static boolean found;
```

Purpose

n

- Size of the chessboard.

board

- Stores queen positions.

found

- Checks whether at least one solution exists.

5. Board Initialization

</> Java



```
board = new int[n][n];
```

Initially

```
0 0 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```



6. isSafe() Function

</> Java



```
static boolean isSafe(int row, int col)
```

Purpose

Checks whether a queen can be placed at

```
(row, col)
```



It checks **three conditions**.

(1) Column Check

</> Java



```
for(int i=0;i<row;i++){
    if(board[i][col]==1)
        return false;
}
```

Checks whether another queen exists in the same column.

Example

```
. Q . .
. Q . .
```



✗ Not Allowed

(2) Left Diagonal Check

Java



```
for(int i=row-1,j=col-1;i>=0 && j>=0;i--,j--)
```

Direction

↖



Example

```
Q . . .  
. ? . .
```



Queen attacks diagonally.

✗ Return false

(3) Right Diagonal Check

Java



```
for(int i=row-1,j=col+1;i>=0 && j<n;i--,j++)
```

Direction

↗



Example

```
. . . Q  
. . ? .
```



✗ Return false

Safe Position

If all checks pass

Java



```
return true;
```

7. solve() Function

</> Java



```
static void solve(int row)
```

Purpose

Places queens row by row using backtracking.

Base Case

</> Java



```
if(row==n)
```

Meaning

All queens are successfully placed.

```
Row 0 ✓  
Row 1 ✓  
Row 2 ✓  
Row 3 ✓
```

```
row = 4
```



Solution found.

Mark Solution

</> Java



```
found=true;
```

Indicates that a valid solution exists.

Print Board

</> Java



```
for(int i=0;i<n;i++){  
    for(int j=0;j<n;j++){  
        System.out.print(board[i][j]);  
    }  
    System.out.println();  
}
```

Example Output

```
0100  
0001  
1000  
0010
```



Try Every Column

</> Java



```
for(int col=0;col<n;col++)
```

For the current row, try placing the queen in every column.

Example

Row 0



Try

Column 0
Column 1
Column 2
Column 3

Check Safe Position

</> Java



```
if(isSafe(row,col))
```

If safe,

Place the queen.

Place Queen

</> Java



```
board[row][col]=1;
```

Example

```
0 1 0 0
0 0 0 0
0 0 0 0
0 0 0 0
```



Recursive Call

</> Java



```
solve(row+1);
```

Move to the next row.

Backtracking Step

</> Java



```
board[row][col]=0;
```

Remove the queen if no solution is possible.

Example

Before Backtracking

```
Q . . .
. . Q .
. . . Q
```



No place for Row 3.

Remove last queen.

```
Q . . .
. . Q .
```



.

Try another column.

8. Main Function

Create Scanner

</> Java



```
Scanner sc = new Scanner(System.in);
```

Used for input.

Number of Test Cases

</> Java



```
int T = sc.nextInt();
```

Example

```
2
4
3
```



```
T = 2
```



Loop Through Test Cases

</> Java



```
for(int l=0;l<T;l++)
```

Runs the program for each test case.

Read N

</> Java



```
n = sc.nextInt();
```

Example

4



Create Board

</> Java



```
board = new int[n][n];
```

Creates an empty chessboard.

Reset Flag

</> Java



```
found = false;
```

Resets solution status for every test case.

Start Solving

</> Java



```
solve(0);
```

Starts placing queens from Row 0.

No Solution

</> Java



```
if(!found) {  
    System.out.println("-1");  
}
```

```
}
```

If no arrangement exists,

Output

```
-1
```



Example

```
N = 2
```

```
N = 3
```



No solution exists.

9. Complete Flow

```
solve(0)
```

```
↓
```

```
Try Column 0
```

```
↓
```

```
Safe?
```

```
↓
```

```
Yes
```

```
↓
```

```
Place Queen
```

```
↓
```

```
solve(1)
```

```
↓
```

```
Try Every Column
```

```
↓
```

```
Safe?
```

```
↓
```



Yes

↓

Place Queen

↓

solve(2)

↓

No Position?

↓

Backtrack

↓

Remove Queen

↓

Try Next Column

↓

Repeat Until Solution Found

10. Time Complexity

- **Time Complexity: $O(N!)$**
 - In the worst case, each row tries many possible columns, leading to factorial growth.
- **Space Complexity: $O(N^2)$** for the board + **$O(N)$** recursion stack.

Quick Revision (Exam Points)

- **Problem:** Place **N queens** so that no two attack each other.
- **Technique Used:** Backtracking.
- **Safe Checks:** Column, Left Diagonal, Right Diagonal.
- **Base Case:** `row == n`.

- **Recursive Step:** `solve(row + 1)` .
- **Backtracking Step:** `board[row][col] = 0` .
- **Time Complexity:** $O(N!)$.
- **Space Complexity:** $O(N^2)$ (board) + $O(N)$ (recursion stack).

These notes are concise and ideal for last-minute revision before exams or coding interviews.